

É bem simples, porém requer atenção, basicamente precisamos de instalar 4 bibliotecas antes, para instalar usando o vsCode você pode usar o terminal com o comando `'dotnet add package [nomeDaBiblioteca] --version [versão compatível com seu .net]'` ou de um jeito bem mais fácil, clique com o botão direito na pasta raiz do projeto e procure pela opção `c# package management`, depois selecione `add nuget package`, agora independente do seu método baixe as 4 bibliotecas: `Microsoft.EntityFrameworkCore` , `Microsoft.EntityFrameworkCore.Design` , `Microsoft.EntityFrameworkCore.SqlServer` , `Microsoft.EntityFrameworkCore.Tools` . Após isso crie uma pasta chamada `DbContext`, nela ficará um arquivo de mesmo nome `.cs`, que terá uma classe do tipo `DbContext`, '`public class ApplicationDbContext : DbContext`' veja 2 exemplos completos:

```
using Microsoft.EntityFrameworkCore;
using ContosoPizza.Models;

namespace ContosoPizza.DataContext
{
    public class ApplicationDbContext : DbContext
    {
        // Esta linha abaixo onde definimos o construtor é de extrema
        // importância, e sempre será utilizada para fazer conexão com o banco
        // de dados.
        public
        ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {

        }
    }
}
```

Nesse primeiro exemplo usamos o método `DbSet` que pega os models (lembre-se de pegar seus models usando um namespace para poder usá-los como uma biblioteca) e os instância como objetos, quando usarmos o `migration`, este método automaticamente irá conectar os models com as tabelas do banco e se as tabelas não existirem, assim como o banco, ele automaticamente irá criar o banco com suas tabelas, porém aqui vai um adendo, se caso for só conectar com um banco já

existente, o nome do objeto tem que ser exatamente igual ao da tabela e seus parâmetros igual em nome e tipo aos das colunas da tabela.

Agora vamos ao segundo exemplo que é bastante usado quando vamos justamente só conectar com o banco, e as vezes os nomes são diferentes ou tem mais ou menos parâmetros que a tabela do banco legado, a primeira parte deste exemplo é quando vamos conectar com o banco, porém nossos parâmetros são exatamente iguais e o nome também é igual:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Clientes>(); //.ToTable("Clientes");
    (Se caso a tabela possuir um nome diferente da classe model.)

    modelBuilder.Entity<ItemPedido>();

    modelBuilder.Entity<Pagamento>();

    modelBuilder.Entity<Pedido>();

    modelBuilder.Entity<Pizza>();
}
```

Lembrando que este é um método que será escrito dentro da nossa classe, agora quando houver diferença nos parâmetros e no nome:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Mapeamento da entidade ClienteModel
    modelBuilder.Entity<ClienteModel>(entity =>
    {
        // ♦ Nome da tabela no banco
        entity.ToTable("Clientes");
    });
}
```

```
// ♦ Chave primária
entity.HasKey(c => c.Id);

// ♦ Mapeamento das colunas
entity.Property(c => c.Id)
    .HasColumnName("Id");

entity.Property(c => c.Nome)
    .HasColumnName("nome_cliente")
```

Pronto agora estamos prontos para o migration.